

SKILL WEEK10

TASK1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

```
GNU nano 8.7.1 task1.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig)
{
    printf("\nSignal received: %d\n", sig);
    printf("Signal handled successfully!\n");
}

int main()
{
    signal(SIGINT, handle_signal);

    printf("Process started.\n");
    printf("PID: %d\n", getpid());
    printf("Press Ctrl+C to send SIGINT.\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }

    return 0;
}
```

```
rishill10@ASUS16X: ~/Skill10  Ubuntu
rishill10@ASUS16X:~$ mkdir Skill10
cd Skill10
rishill10@ASUS16X:~/Skill10$ nano task1.c
rishill10@ASUS16X:~/Skill10$ gcc task1.c -o task1
./task1
Process started.
PID: 1354
Press Ctrl+C to send SIGINT.
Program is running...
Program is running...
Program is running...
^C
Signal received: 2
Signal handled successfully!
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Killed ./task1
rishill10@ASUS16X:~/Skill10$
```

TASK2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

```
GNU nano 8.7.1 task2.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sigint_handler(int sig)
{
    printf("\nSIGINT received!\n");
    printf("The program received Ctrl+C.\n");
}

int main()
{
    signal(SIGINT, sigint_handler);

    printf("Program PID: %d\n", getpid());
    printf("Press Ctrl+C to test SIGINT.\n");

    while (1)
    {
        printf("Foreground job is running...\n");
        sleep(3);
    }
    return 0;
}
```

```
rishill10@ASUS16X: ~/Skill10  x  Ubuntu
rishill10@ASUS16X:~/Skill10$ nano task2.c
rishill10@ASUS16X:~/Skill10$ gcc task2.c -o task2
./task2
Program PID: 1492
Press Ctrl+C to test SIGINT.
Foreground job is running...
^C
SIGINT received!
The program received Ctrl+C.
Foreground job is running...
Foreground job is running...
Foreground job is running...
^Z
[1]+  Stopped                  ./task2
rishill10@ASUS16X:~/Skill10$ jobs
[1]+  Stopped                  ./task2
rishill10@ASUS16X:~/Skill10$ bg %1
[1]+ ./task2 &
Foreground job is running...
rishill10@ASUS16X:~/Skill10$ jobs
[1]+  Running                  ./task2 &
rishill10@ASUS16X:~/Skill10$
Foreground job is running...
Foreground job is running...
Foreground job is running...
```

TASK3: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

```
GNU nano 8.7.1 task3.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sigtstp_handler(int sig)
{
    printf("\nSIGTSTP received!\n");
    printf("Process received Ctrl+Z.\n");
}

int main()
{
    signal(SIGTSTP, sigtstp_handler);

    printf("Process PID: %d\n", getpid());
    printf("Press Ctrl+Z to send SIGTSTP.\n");

    while (1)
    {
        printf("Process is running...\n");
        sleep(2);
    }
    return 0;
}
```

```
rishill10@ASUS16X: ~/Skill10
rishill10@ASUS16X:~/Skill10$ nano task3.c
rishill10@ASUS16X:~/Skill10$ gcc task3.c -o task3
./task3
Process PID: 1630
Press Ctrl+Z to send SIGTSTP.
Process is running...
Process is running...
^Z
SIGTSTP received!
Process received Ctrl+Z.
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Terminated ./task3
rishill10@ASUS16X:~/Skill10$
```

TASK4: To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```
GNU nano 8.7.1 task4.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid = getpid();
    pid_t ppid = getppid();
    pid_t pgid = getpgrp();
    pid_t sid = getsid(0);

    printf("Process ID (PID): %d\n", pid);
    printf("Parent PID: %d\n", ppid);
    printf("Process Group ID (PGID): %d\n", pgid);
    printf("Session ID (SID): %d\n", sid);

    printf("\nProcess is running...\n");

    while (1)
    {
        sleep(2);
    }
    return 0;
}
```

```
rishill10@ASUS16X: ~/Skill10
rishill10@ASUS16X:~/Skill10$ nano task4.c
rishill10@ASUS16X:~/Skill10$ gcc task4.c -o task4
./task4
Process ID (PID): 1693
Parent PID: 1128
Process Group ID (PGID): 1693
Session ID (SID): 1128

Process is running...
^Z
[1]+  Stopped                  ./task4
rishill10@ASUS16X:~/Skill10$ jobs
[1]+  Stopped                  ./task4
rishill10@ASUS16X:~/Skill10$ ps -o pid,ppid,pgid,sid,tpgid,stat,cmd
  PID   PPID   PGID   SID   TPGID  STAT  CMD
  1128   1126   1128   1128   1713   Ss    -bash
  1693   1128   1693   1128   1713   T     ./task4
  1713   1128   1713   1128   1713   R+    ps -o pid,ppid,pgid,sid,tpgid,s
rishill10@ASUS16X:~/Skill10$ bg %1
[1]+ ./task4 &
rishill10@ASUS16X:~/Skill10$ jobs
[1]+  Running                  ./task4 &
rishill10@ASUS16X:~/Skill10$ fg %1
./task4
^C
rishill10@ASUS16X:~/Skill10$ |
```